

3. Implement Greedy search algorithm for Selection Sort

Function for Selection Sort

```
def selection_sort(arr):
```

```
    n = len(arr)
```

```
    for i in range(n):
```

```
        # Assume current index has minimum value
```

```
        min_index = i
```

```
        # Find the minimum element
```

```
        for j in range(i + 1, n):
```

```
            if arr[j] < arr[min_index]:
```

```
                min_index = j
```

```
        # Swap minimum element with current element
```

```
        arr[i], arr[min_index] = arr[min_index], arr[i]
```

```
    return arr
```

```
# Main Program
```

```
arr = [64, 25, 12, 22, 11]
```

```
print("Original Array:", arr)
```

```
sorted_arr = selection_sort(arr)
```

```
print("Sorted Array:", sorted_arr)
```

Simple Explanation

- Selection Sort works using a **Greedy approach**.
- In every step, it selects the **smallest element** from the unsorted part.
- Then it places that element in the correct position.
- This process repeats until the array is sorted.

1. Implement depth first search algorithm and Breadth First Search algorithm, Use an undirected graph and develop a recursive algorithm for searching all the vertices of a graph or tree data structure.

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['A', 'D', 'E'],  
    'C': ['A', 'F'],  
    'D': ['B'],  
    'E': ['B'],  
    'F': ['C']  
}
```

```
# DFS using Recursion
```

```
visited = set()
```

```
def dfs(node):
```

```
    if node not in visited:
```

```
        print(node, end=" ")
```

```
        visited.add(node)
```

```
        for neighbor in graph[node]:
```

```
            dfs(neighbor)
```

```

# BFS using Queue
def bfs(start):
    visited = []
    queue = [start]

    visited.append(start)

    while queue:
        node = queue.pop(0)
        print(node, end=" ")

        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.append(neighbor)
                queue.append(neighbor)

# Main Program
print("DFS Traversal:")
dfs('A')
print("\n")
print("BFS Traversal:")
bfs('A')

```

10. Develop an elementary chatbot for any suitable customer interaction application. IFB Washing Machine Service Centre.

```
# Advanced Chatbot - IFB Washing Machine Service Centre
```

```
print("=====")
```

```
print(" Welcome to IFB Service Centre ")
```

```
print("=====")
```

```
name = input("Enter Your Name: ")
```

```
print("\nHello", name + "!!")
```

```
print("How can I help you today?")
```

```
print("Type 'exit' to stop.\n")
```

```
while True:
```

```
    user = input("You: ").lower()
```

```
    # Greeting
```

```
    if user in ["hi", "hello", "hey"]:
```

```
        print("Bot: Hello", name + "!! Welcome to IFB Service Centre.")
```

```
    # Service Booking
```

```
    elif "service" in user or "repair" in user:
```

```
        machine = input("Bot: Which machine do you have? (Top Load / Front Load): ")
```

```
        problem = input("Bot: Enter the problem: ")
```

```
        print("Bot: Your service request has been registered.")
```

```
print("Bot: Machine Type:", machine)
print("Bot: Problem:", problem)
print("Bot: Technician will contact you soon.")
```

```
# Complaint
```

```
elif "complaint" in user:
```

```
    complaint = input("Bot: Please enter your complaint: ")
    print("Bot: Complaint registered successfully.")
```

```
# Price
```

```
elif "price" in user or "charge" in user:
```

```
    print("Bot: Service charges start from Rs. 300.")
```

```
# Location
```

```
elif "location" in user or "address" in user:
```

```
    print("Bot: IFB Service Centre, Pune, Maharashtra.")
```

```
# Contact
```

```
elif "contact" in user or "number" in user:
```

```
    print("Bot: Contact Number: 9876543210")
```

```
# Timing
```

```
elif "time" in user or "timing" in user:
```

```
    print("Bot: Service timing is 9 AM to 6 PM.")
```

```
# Thank You
```

```
elif "thank" in user:
```

```

    print("Bot: Thank you", name + "!")

# Exit
elif user == "exit":
    print("Bot: Thank you for visiting IFB Service Centre.")
    break

# Default
else:
    print("Bot: Sorry! I did not understand your message.")

```

9. Develop an elementary chatbot for any suitable customer interaction application. Blink –IT

Simple Chatbot - Blinkit Customer Support

```

print("=====")
print(" Welcome to Blinkit Chatbot ")
print("=====")

name = input("Enter Your Name: ")

print("\nHello", name + "!")
print("Type 'exit' to stop the chatbot.\n")

while True:

    user = input("You: ").lower()

```

Greeting

if user in ["hi", "hello", "hey"]:

print("Bot: Hello", name + "! How can I help you?")

Order Status

elif "order" in user:

order_id = input("Bot: Enter your Order ID: ")

print("Bot: Your order", order_id, "is out for delivery.")

Delivery Time

elif "delivery" in user:

print("Bot: Delivery usually takes 10 to 20 minutes.")

Payment

elif "payment" in user:

print("Bot: We support UPI, Debit Card, Credit Card and Cash on Delivery.")

Refund

elif "refund" in user:

print("Bot: Refund will be processed within 3 to 5 working days.")

Cancel Order

elif "cancel" in user:

print("Bot: Your order cancellation request has been submitted.")

Contact

elif "contact" in user or "number" in user:

```

print("Bot: Blinkit Customer Care Number: 9876543210")

# Location
elif "location" in user or "address" in user:

    print("Bot: Blinkit service is available in major cities.")

# Thank You
elif "thank" in user:

    print("Bot: Thank you", name + "! Have a nice day.")

# Exit
elif user == "exit":

    print("Bot: Thank you for using Blinkit Chatbot.")

    break

# Default
else:

    print("Bot: Sorry! I did not understand.")

```

8. Develop an elementary chatbot for any suitable customer interaction application. RED-BUS

```

print("=====")

print(" Welcome to RedBus Chatbot ")

print("=====")

name = input("Enter Your Name: ")

```



```
print("\nHello", name + "!!")
```

```
print("Type 'exit' to stop the chatbot.\n")
```

```
while True:
```

```
    user = input("You: ").lower()
```

```
    # Greeting
```

```
    if user in ["hi", "hello", "hey"]:
```

```
        print("Bot: Hello", name + "!! How can I help you?")
```

```
    # Bus Booking
```

```
    elif "book" in user or "ticket" in user:
```

```
        source = input("Bot: Enter Source City: ")
```

```
        destination = input("Bot: Enter Destination City: ")
```

```
        print("Bot: Bus ticket booking from", source, "to", destination, "is available.")
```

```
    # Bus Timing
```

```
    elif "time" in user or "timing" in user:
```

```
        print("Bot: Buses are available 24 hours.")
```

```
    # Fare
```

```
    elif "fare" in user or "price" in user:
```

```
        print("Bot: Ticket price starts from Rs. 500.")
```

```
    # Cancel Ticket
```

```
elif "cancel" in user:
```

```
    print("Bot: Your ticket cancellation request has been submitted.")
```

```
# Refund
```

```
elif "refund" in user:
```

```
    print("Bot: Refund will be processed within 5 working days.")
```

```
# Contact
```

```
elif "contact" in user or "number" in user:
```

```
    print("Bot: RedBus Customer Care Number: 9876543210")
```

```
# Location
```

```
elif "location" in user or "address" in user:
```

```
    print("Bot: RedBus service is available across India.")
```

```
# Thank You
```

```
elif "thank" in user:
```

```
    print("Bot: Thank you", name + "! Have a safe journey.")
```

```
# Exit
```

```
elif user == "exit":
```

```
    print("Bot: Thank you for using RedBus Chatbot.")
```

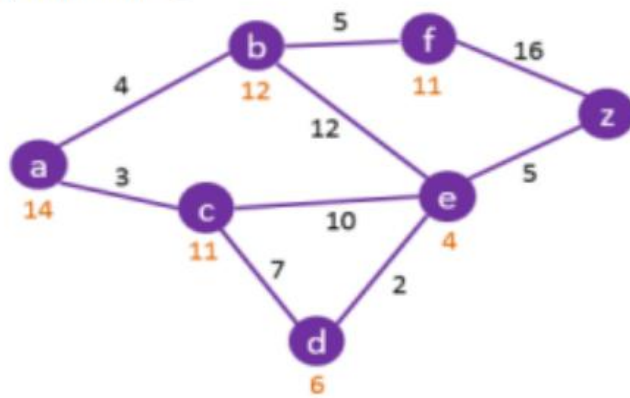
```
    break
```

```
# Default
```

```
else:
```

```
    print("Bot: Sorry! I did not understand your message.")
```

11. Implement A* Algorithm for Graph search (Minimum cost) (Assume suitable graph structure.)



graph = {

'a': [('b',4), ('c',3)],

'b': [('e',12), ('f',5)],

'c': [('d',7), ('e',10)],

'd': [('e',2)],

'e': [('z',5)],

'f': [('z',16)],

'z': []

}

h = {

'a':14,

'b':12,

'c':11,

'd':6,

'e':4,

'f':11,

'z':0

```
}
```

```
def a_star(start, goal):
```

```
    open_list = [(start, 0)]
```

```
    visited = []
```

```
    while open_list:
```

```
        # Find minimum f(n)
```

```
        current, cost = min(open_list,
```

```
                             key=lambda x: x[1] + h[x[0]])
```

```
        open_list.remove((current, cost))
```

```
        print("Visited:", current)
```

```
        if current == goal:
```

```
            print("Goal Reached")
```

```
            print("Minimum Cost =", cost)
```

```
            break
```

```
        visited.append(current)
```

```
        # Add neighbors
```

```
        temp = []
```

```

    for neigh, weight in graph[current]:

        if neigh not in visited:

            temp.append((neigh, cost + weight))

        # Keep only best nodes

    open_list = temp

# Main
a_star('a', 'z')

```

14. Implement Greedy search algorithm

```

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['G'],
    'F': ['G'],
    'G': []
}

# Heuristic values
h = {
    'A': 5,
    'B': 4,

```

```
'C': 2,  
'D': 6,  
'E': 3,  
'F': 1,  
'G': 0  
}
```

```
def greedy(start, goal):
```

```
    open_list = [start]
```

```
    visited = []
```

```
    while open_list:
```

```
        # Select node with minimum heuristic value
```

```
        current = min(open_list, key=lambda x: h[x])
```

```
        open_list.remove(current)
```

```
        print("Visited:", current)
```

```
        if current == goal:
```

```
            print("Goal Reached")
```

```
            break
```

```
        visited.append(current)
```

```
# Add neighbors

for neigh in graph[current]:

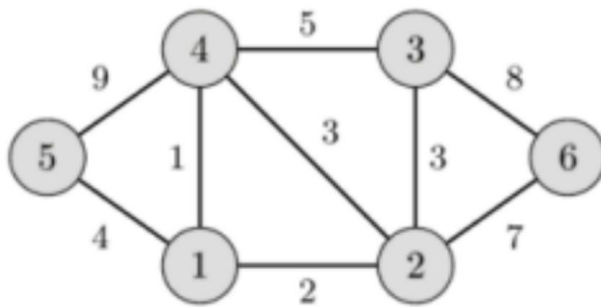
    if neigh not in visited:

        open_list.append(neigh)
```

```
# Main

greedy('A', 'G')
```

4. Implement Greedy search algorithm for any of the following application: Prim's Minimal Spanning Tree Algorithm (Consider Starting index as 1)



Prim's Minimum Spanning Tree Algorithm

```
graph = [
#1 2 3 4 5 6
[0,2,0,1,4,0], # 1
[2,0,3,3,0,7], # 2
[0,3,0,5,0,8], # 3
[1,3,5,0,9,0], # 4
[4,0,0,9,0,0], # 5
[0,7,8,0,0,0] # 6
]
```

```
v = 6
```

```
selected = [False] * v
```

```
# Start from vertex 1
```

```
selected[0] = True
```

```
print("Edge : Weight")
```

```
for i in range(v - 1):
```

```
    minimum = 999
```

```
    x = 0
```

```
    y = 0
```

```
    for j in range(v):
```

```
        if selected[j]:
```

```
            for k in range(v):
```

```
                if not selected[k] and graph[j][k]:
```

```
                    if minimum > graph[j][k]:
```

```
                        minimum = graph[j][k]
```



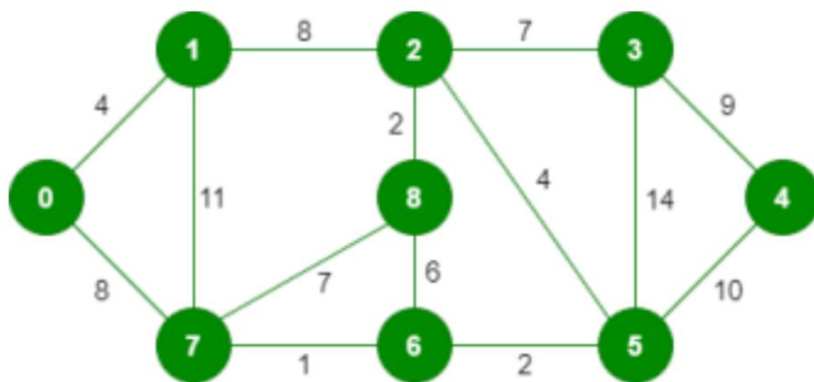
```
x = j
```

```
y = k
```

```
print(x + 1, "-", y + 1, ":", graph[x][y])
```

```
selected[y] = True
```

5. Implement Greedy search algorithm for any of the following application: Kruskal's Minimal Spanning Tree Algorithm



```
# Kruskal's Minimum Spanning Tree Algorithm
```

```
# Edges -> (weight, vertex1, vertex2)
```

```
edges = [  
    (4, 0, 1),  
    (8, 0, 7),  
    (11, 1, 7),  
    (8, 1, 2),  
    (7, 2, 3),  
    (2, 2, 8),  
    (4, 2, 5),
```

```
(9, 3, 4),  
(14, 3, 5),  
(10, 4, 5),  
(2, 5, 6),  
(1, 6, 7),  
(6, 6, 8),  
(7, 7, 8)  
]
```

```
# Sort edges by weight
```

```
edges.sort()
```

```
parent = []
```

```
# Create parent array
```

```
for i in range(9):
```

```
    parent.append(i)
```

```
# Find function
```

```
def find(i):
```

```
    while parent[i] != i:
```

```
        i = parent[i]
```

```
    return i
```

```
# Union function
```

```

def union(x, y):
    parent[x] = y

print("Edge : Weight")

# Kruskal Algorithm
for weight, u, v in edges:

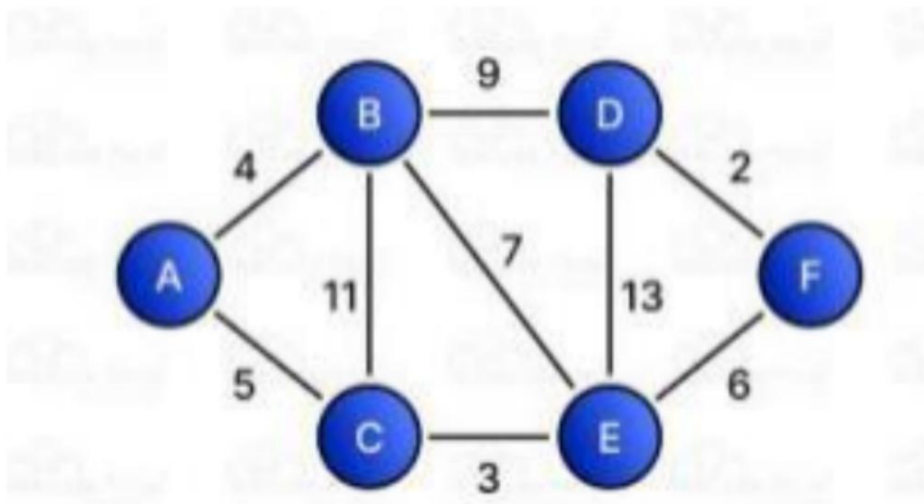
    x = find(u)
    y = find(v)

    # Avoid cycle
    if x != y:

        print(u, "-", v, ":", weight)
        union(x, y)

```

6. Implement Greedy search algorithm for any of the following application: Dijkstra's Algorithm.



```
# Dijkstra's Algorithm
```

```
graph = {  
    'A': {'B':4, 'C':5},  
    'B': {'A':4, 'C':11, 'D':9, 'E':7},  
    'C': {'A':5, 'B':11, 'E':3},  
    'D': {'B':9, 'E':13, 'F':2},  
    'E': {'B':7, 'C':3, 'D':13, 'F':6},  
    'F': {'D':2, 'E':6}  
}
```

```
# Distance dictionary
```

```
distance = {}
```

```
# Initialize distances
```

```
for node in graph:
```

```
    distance[node] = 999
```

```
# Starting node
```

```
start = 'A'
```

```
distance[start] = 0
```

```
visited = []
```

```
while len(visited) < len(graph):
```

```
    # Find minimum distance node
```

```
min_node = None
```

```
for node in graph:
```

```
    if node not in visited:
```

```
        if min_node is None or distance[node] < distance[min_node]:
```

```
            min_node = node
```

```
visited.append(min_node)
```

```
# Update distances
```

```
for neighbor, weight in graph[min_node].items():
```

```
    if distance[min_node] + weight < distance[neighbor]:
```

```
        distance[neighbor] = distance[min_node] + weight
```

```
# Print shortest distances
```

```
print("Shortest Distance from A")
```

```
for node in distance:
```

```
    print(node, "=", distance[node])
```

```
# N-Queens Problem using Backtracking
```

```
n = int(input("Enter N: "))
```

```
board = []
```

```
# Function to place queens
```

```
def queen(row):
```

```
    # All queens placed
```

```
    if row == n:
```

```
        print("\nSolution:")
```

```
        for i in range(n):
```

```
            print("Row", i, "-> Column", board[i])
```

```
        return True
```

```
# Try all columns
```

```
for col in range(n):
```

```
    # Check safe position
```

```
    if all(col != board[i] and
```

```
        abs(col - board[i]) != row - i
```

```
        for i in range(row)):
```

```
# Place queen  
board.append(col)
```

```
# Recursive call  
if queen(row + 1):  
    return True
```

```
# Backtracking  
board.pop()
```

```
# Main  
queen(0)
```